
Forum Avancé

Bienvenue dans la version avancée du Forum. Il s'agira de développer pour notre client une **application web complète de forum de discussion**, avec modération, intelligence artificielle et infrastructure de production. Le projet est en état de développement de la MVP.

La deadline est serrée, donc vous devrez optimiser votre temps en travaillant intelligemment.

Je tiens à rappeler ici qu'il ne s'agit pas d'un projet scolaire. Mettez-vous en condition : vous êtes déjà freelance.

Technologies

Front-end : React.js

- Tanstack/Query
- Tanstack/Router
- Tailwind CSS
- Shadcn/ui
- React Hook Form + Zod
- Zustand

Back-end : Golang

- API REST (respecter la norme REST)
- Documentation Swagger (ou Postman, au choix)
- ORM obligatoire avec gestion des migrations

Intelligence Artificielle

- Utiliser un LLM au choix : **OpenAI**, **Anthropic (Claude)**, ou **Ollama** (local / gratuit)
 - Ollama permet de faire tourner des modèles en local sans clé API ni coût

Message Broker

- **RabbitMQ** (via Docker) pour le traitement asynchrone de la modération IA

Base de données

-
- Relationnelle, hébergée en production (Neon, PlanetScale, Supabase, ou autre)
-

Équipe et organisation

- 2 à 3 personnes
- Un compte rendu devra impérativement être effectué au consultant (l'intervenant) en cas de difficulté mettant en danger le projet (problème équipe, incompatibilité technique, deadline, ...). Et ce au plus tôt.

IA

IA autorisée avec intelligence. Tout le code produit devra être **relu, vérifié, et testé manuellement**.

Déploiement en production

- Le front sera déployé sur un **CDN** de votre choix (Netlify, Cloudflare, ...)
- Le back sera conteneurisé dans une **image Docker** stockée sur Docker Hub
- La DB sera hébergée en production (Neon, PlanetScale, Supabase, ou autre)

Docker Swarm (scaling local)

L'infrastructure back-end devra être orchestrée via **Docker Swarm**. Vous devrez démontrer en local la capacité de votre architecture à **scaler horizontalement** :

- Définir un `docker-compose.yml` compatible Swarm (avec `deploy.replicas`)
 - Démonstration du scaling : augmenter/diminuer le nombre de réplicas du service API
 - Load balancing automatique entre les réplicas
 - Les requêtes doivent être réparties correctement (démonstration lors de l'oral)
-

Fonctionnalités (dev)

- Mettre en place **Sentry** pour le tracking des erreurs en production. Vous veillerez à ce que les source maps soient bien configurées.

-
- La **CI/CD** (technologie au choix, devra être motivée) sera déclenchée automatiquement lors d'un push sur la branche main.

- **Back-end** : build automatique de l'image Docker, puis publication automatique sur **Docker Hub**
 - **Front-end** : déploiement automatique sur le CDN choisi
 - Les **migrations** de base de données doivent se faire en automatique
-

Fonctionnalités (documentation)

- Un **Swagger** de l'API sera produit
 - Un **MCD** de la base de données sera produit
 - Une **maquette faible résolution** (type zoning/wireframe) sera produite. Doit être réalisée au plus vite et fournie au consultant.
 - Vous mettrez en place tous les systèmes et modèles de gestion de projet que vous jugerez utiles à la bonne mise en œuvre du projet.
-

Fonctionnalités (user)

Authentification

Deux méthodes d'authentification doivent être proposées :

A) Email / mot de passe

- Inscription avec : email, nom d'utilisateur, mot de passe
 - Le mot de passe doit être **chiffré** avant stockage (bcrypt)
- L'inscription doit intégrer un **captcha** (anti-bot)
- **Mot de passe oublié** : l'utilisateur peut demander la réinitialisation de son mot de passe. Un **email contenant un lien unique et temporaire** lui est envoyé.

B) OAuth (Google et/ou GitHub)

- L'utilisateur peut s'inscrire et se connecter via un fournisseur OAuth
- Au minimum **un provider** doit être implémenté (Google ou GitHub)

Unicité du compte par email (fusion)

L'adresse email est la **clé d'unicité** d'un compte utilisateur. Un email = un compte, quelle que soit la méthode de connexion. Scénarios attendus :

-
- **Scénario 1** : un utilisateur s'inscrit via email/mot de passe avec `alice@mail.com`. Plus tard, il clique sur « Se connecter avec Google » avec le même email. Le système détecte qu'un compte existe déjà : il rattache le provider Google au compte existant et connecte l'utilisateur sur son compte habituel (même ID, mêmes posts, même historique).
 - **Scénario 2** : un utilisateur s'inscrit via Google avec `bob@mail.com`. Un compte est créé automatiquement. Il pourra ensuite, depuis son profil, définir un mot de passe pour activer aussi la connexion classique.
 - **Principe général** : un utilisateur = un email = un ID. Les différentes méthodes de connexion (email/password, Google, GitHub) sont des **moyens d'accès** rattachés à un compte unique, pas des comptes distincts.

Forum

- Créer, éditer et supprimer ses posts (avec éditeur de texte enrichi)
- Associer des catégories aux posts
- Commenter les posts
- Liker / disliker les posts et commentaires
- **Upload d'images** dans les posts (PNG, JPEG, GIF — max 20 MB)
 - Les images doivent être stockées dans une **solution compatible S3** (MinIO, AWS S3, Azure Blob Storage, ...)
- Page de profil / activité

Intelligence Artificielle

- **Aide à la rédaction** : lors de la création d'un post, l'utilisateur peut demander à l'IA de l'aider à reformuler, corriger, enrichir ou compléter son texte. L'intégration doit se faire de manière fluide dans l'éditeur enrichi (ex : bouton « Améliorer avec l'IA », suggestion inline, etc.).
- **Moteur de recherche par IA** : recherche sémantique dans les posts du forum. L'utilisateur peut poser une question en langage naturel et obtenir les posts les plus pertinents.

Notifications

- **Temps réel (WebSocket)** : l'utilisateur reçoit des notifications en direct lorsqu'on commente, like ou répond à l'un de ses posts.
- **Web Push** : même si l'utilisateur n'est pas sur le site, il reçoit une notification push dans son navigateur (via l'API Push + Service Worker). L'utilisateur doit pouvoir gérer

ses préférences de notification.

Messagerie privée

- Les utilisateurs peuvent s'envoyer des **messages privés** en temps réel (WebSocket)
 - Historique des conversations persisté en base
-

Fonctionnalités (admin)

- Dashboard d'administration
- CRUD sur les **catégories**
- Supprimer / masquer des posts et commentaires
- Promouvoir ou rétrograder un utilisateur en **modérateur**
- Visualiser les comptes utilisateurs
- **Système de signalement** : les utilisateurs signalent du contenu, les modérateurs traitent, l'admin arbitre

Modération automatique par IA (via RabbitMQ)

En complément de la modération humaine, un système de **modération automatique par IA** doit analyser le contenu des posts et commentaires de manière **asynchrone** via une architecture à base de message queue :

Architecture attendue :

- Lorsqu'un post ou commentaire est publié, l'API publie un message dans une **queue RabbitMQ** (`moderation.check`) contenant l'ID et le contenu à analyser
- Un **worker dédié** consomme la queue, appelle le LLM pour analyser le contenu, et met à jour le statut du post en base de données. Ce worker est un **2e backend Golang indépendant** (binaire et image Docker séparés de l'API principale)
- Le post est publié immédiatement avec un statut `pending_moderation`, puis passe à `approved`, `flagged` ou `blocked` selon le verdict de l'IA
- Cette architecture découple l'API du temps de réponse du LLM et permet de scaler les workers indépendamment dans Docker Swarm

Fonctionnalités de modération :

- Détection de contenu inapproprié (insultes, spam, contenu illégal, etc.)

-
- Le contenu détecté peut être : bloqué automatiquement, mis en attente de validation par un modérateur, ou flagé avec un avertissement
 - Un **dashboard de modération IA** doit permettre aux admins de visualiser les décisions de l'IA et de les corriger si nécessaire

Stress test :

- La capacité du système de modération à encaisser la charge devra être **démontrée via un stress test** (par exemple : **Artillery**, k6, ou équivalent)
- Le test devra simuler un volume important de publications simultanées et vérifier que la queue RabbitMQ absorbe correctement la charge sans perte de messages
- Les résultats du stress test (débit, latence, taux d'erreur) devront être présentés lors de l'oral

Tests d'intégration IA :

Les décisions de l'IA de modération doivent être **testées via des tests d'intégration** afin de garantir sur la durée que son comportement reste correct :

- Constituer un **dataset de messages de test** couvrant différents cas : messages normaux, insultes, spam, contenu illégal, cas limites, etc.
- Chaque message du dataset doit avoir un **verdict attendu** (approved, flagged, blocked)
- Les tests vérifient que la décision rendue par l'IA correspond au verdict attendu
- Ces tests doivent être **intégrés à la CI** pour détecter toute régression (ex : changement de modèle, de prompt, ou de provider)

Sécurisation

- L'authentification et l'accès à l'API seront sécurisés par **JWT** (access token + refresh token)
- L'env sera géré de manière à ne pas exposer les secrets
- **Rate limiting** sur les endpoints sensibles

L'oral

Changement de règle par rapport aux autres oraux. Celui-ci sera en 2 parties :

-
- Une **partie commerciale vulgarisée** où vous appuierez les qualités de votre production (public non technique).
 - Une **partie technique** où vous décortiquerez votre stack. Vous devrez expliquer vos choix, vos méthodes et les défendre en démontrant que vous connaissez votre sujet (public professionnel du développement). La vulgarisation est ici à oublier. Vous utiliserez tous les termes techniques de rigueur.
-

Note

Histoire de rassurer : la note Ynov sera basée sur les attendus d'une première année.

En résumé, même avec 20% de ce qui est demandé dans ce sujet, vous êtes déjà assuré de largement valider votre module.